

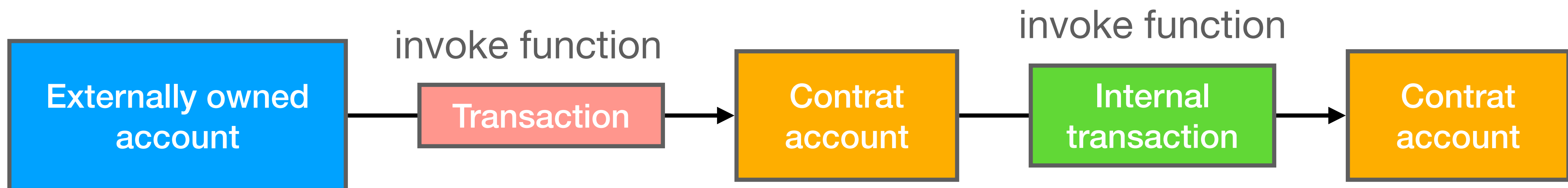
Smart Contract security

Leander Jehl

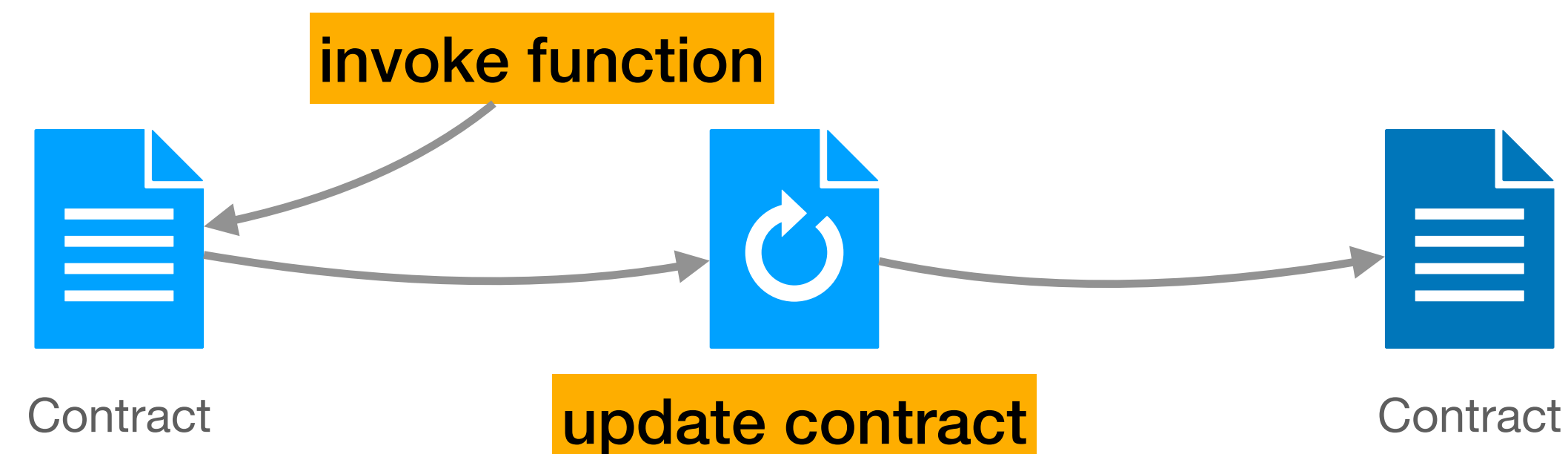
Recap

Smart Contracts

Transactions are used to transfer ether, invoke functions, and deploy new contracts.



Think of a Smart Contract like an object from OOP with fields and methods



**State stored in Ethereum
blockchain**

Bitcoin

Block structure

Header:

PrevBlockhash
Nonce
Timestamp

Transaction data

Merkle tree

Merkle tree allows to easily proof that a transaction is included in a block.

State of the blockchain (UTXO)
is not in the block.

Ethereum

Block structure

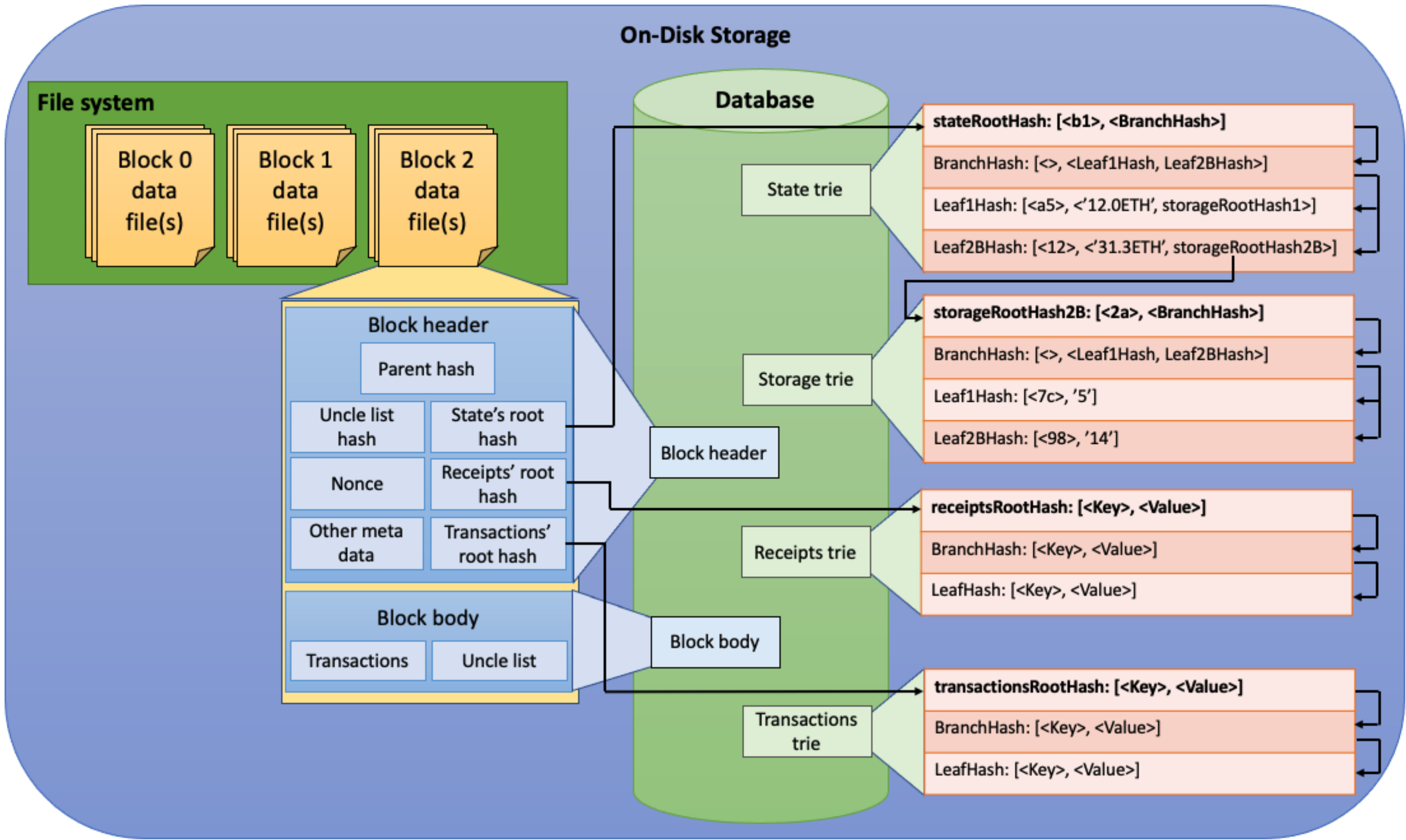
Header:

PrevBlockhash
Nonce
Timestamp

State root hash
Receipts root hash

Transaction data

Merkle tree



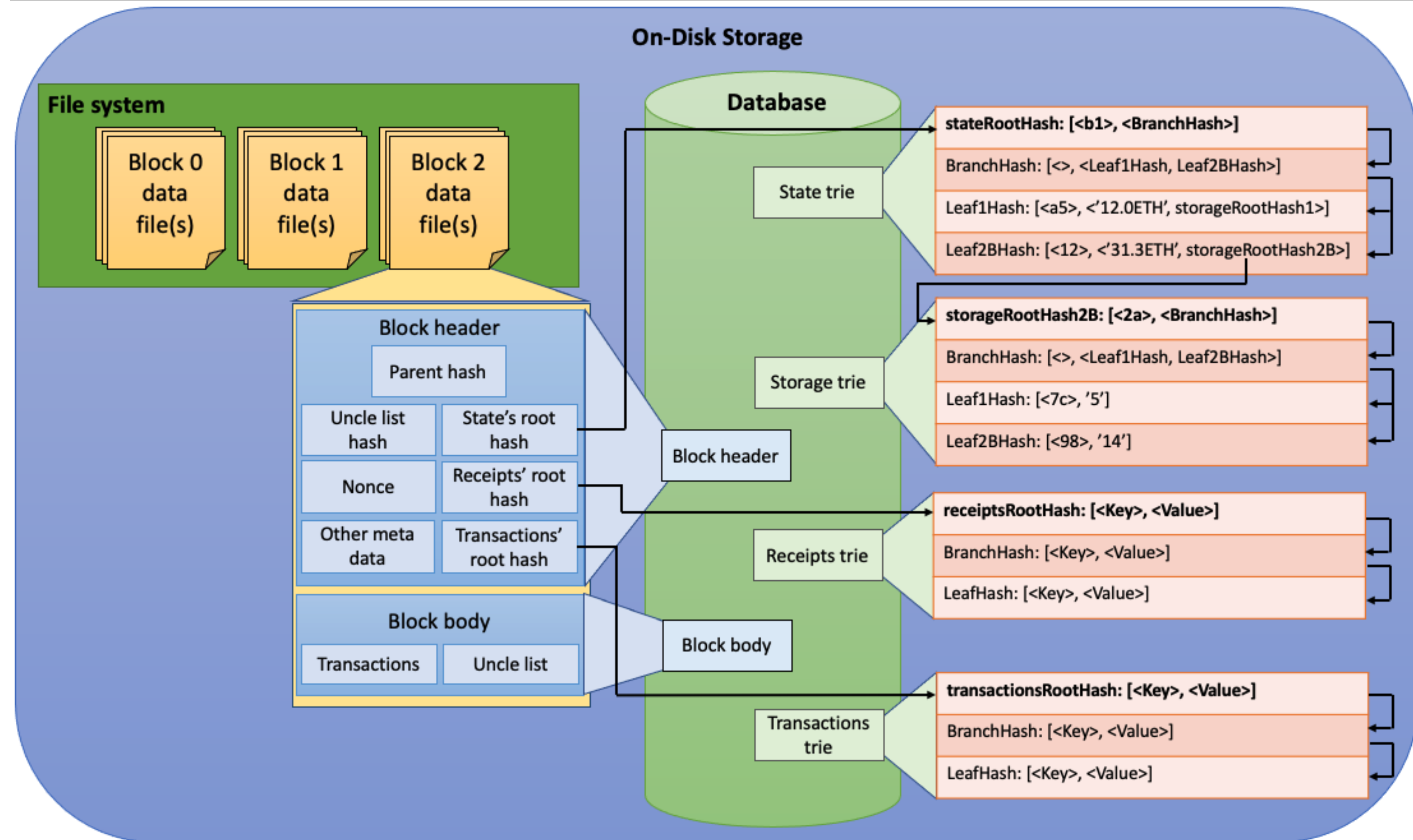
Ethereum

Receipts trie

Stores transaction results:

```
From: address  
To: address  
Status: ... // aborted?  
Logs: events  
ContractAddress address  
// new contract address,  
// if created
```

Return transaction results,
by emitting Events,
which are added to the logs.



Ethereum

State trie

Stores accounts:

Account	
A	“ad1b...”
B	“ad92...”
C	“53e2...”

Address:
[Value,
 Nonce,
 StorageRoot,
 CodeHash]

Trie:
Merkle tree that supports

update
lookup
proof

Ethereum

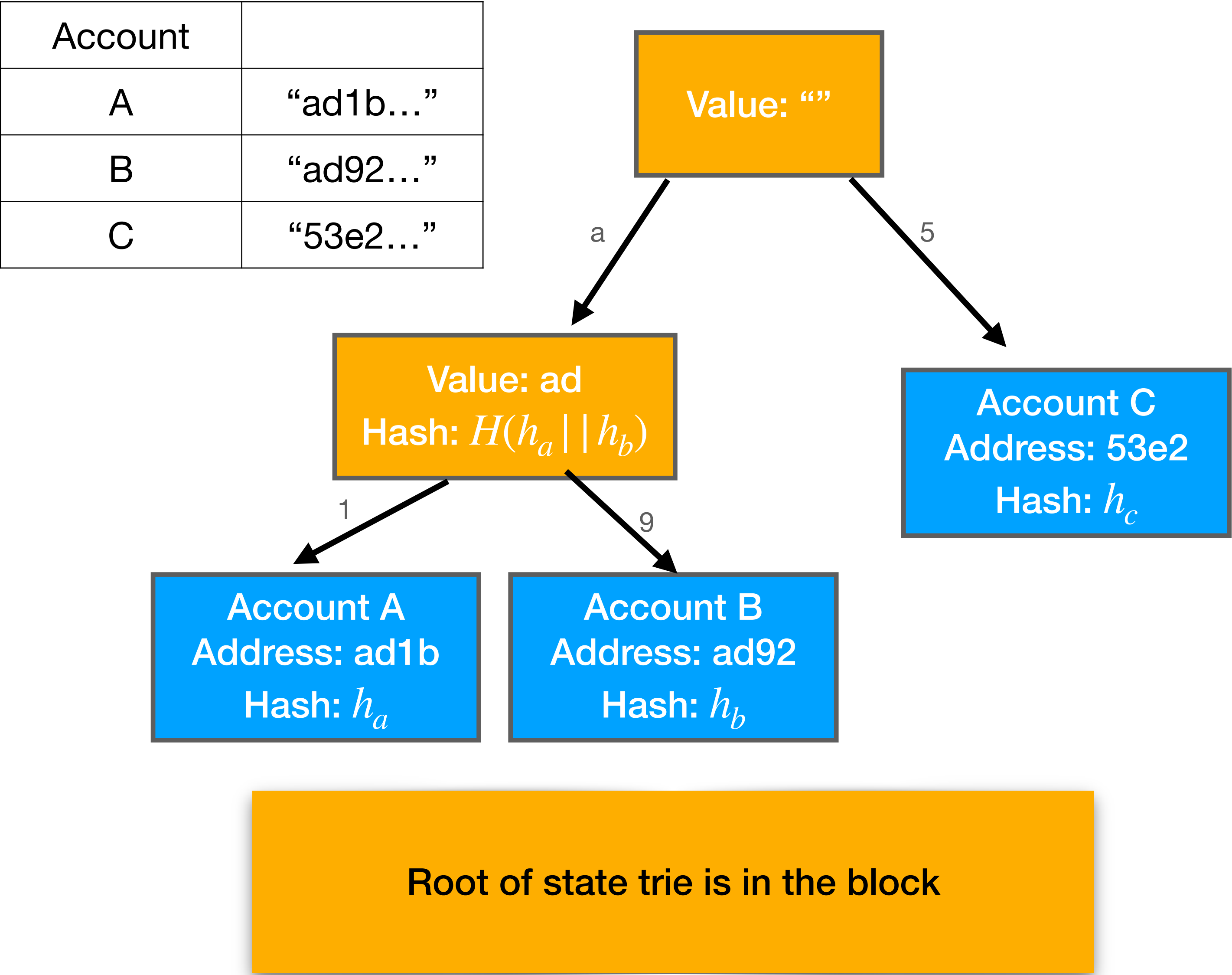
State trie

Stores accounts:

Address:
[Value,
Nonce,
StorageRoot,
CodeHash]

Trie:
Merkle tree that supports

update
lookup
proof



Ethereum

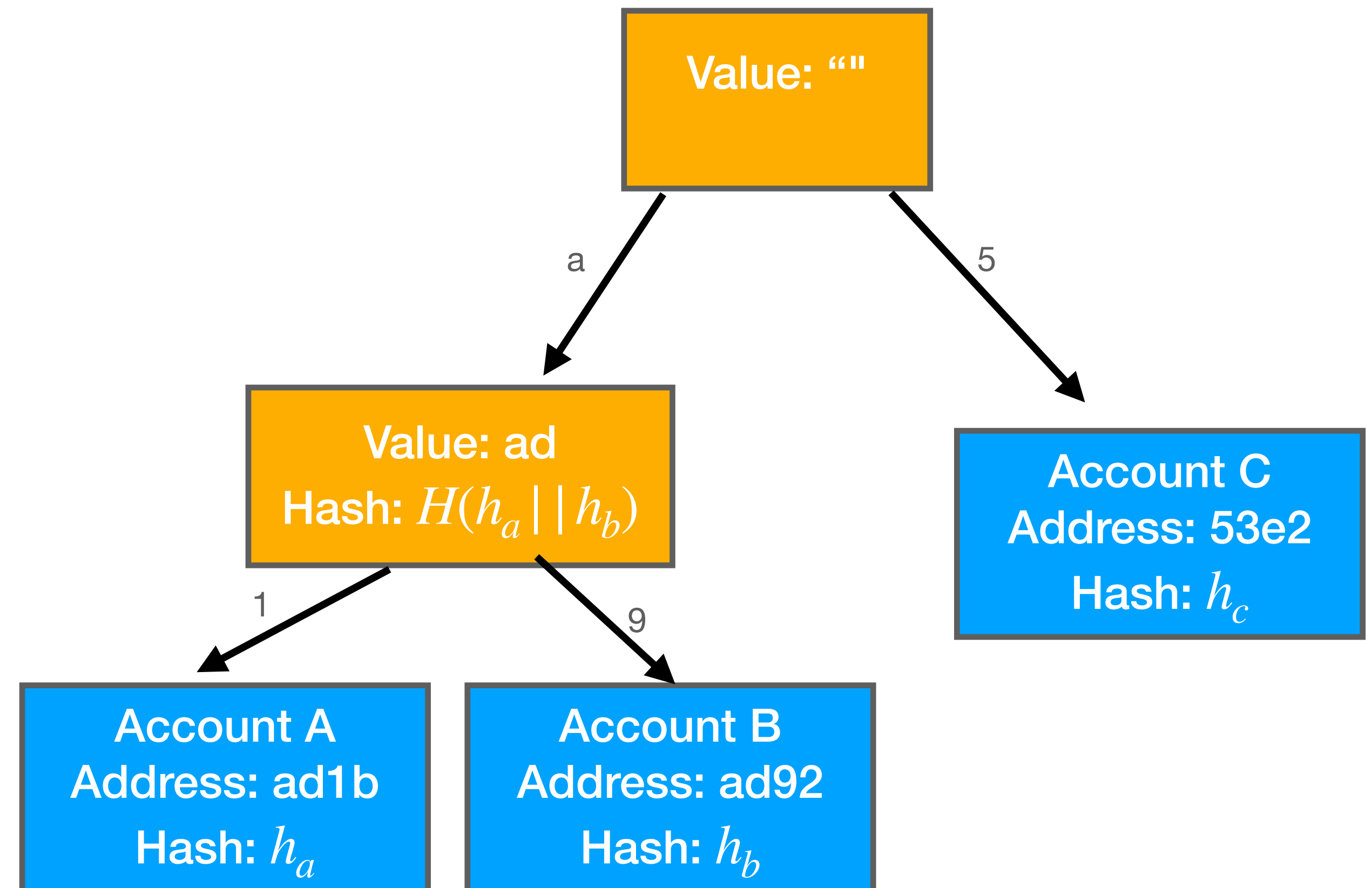
State trie

Stores accounts:

Address:
[Value,
Nonce,
StorageRoot,
CodeHash]

Trie:
Merkle tree that supports

update
lookup
proof



If account Value changes
-> SC Account hash changes
-> State trie root changes

Ethereum

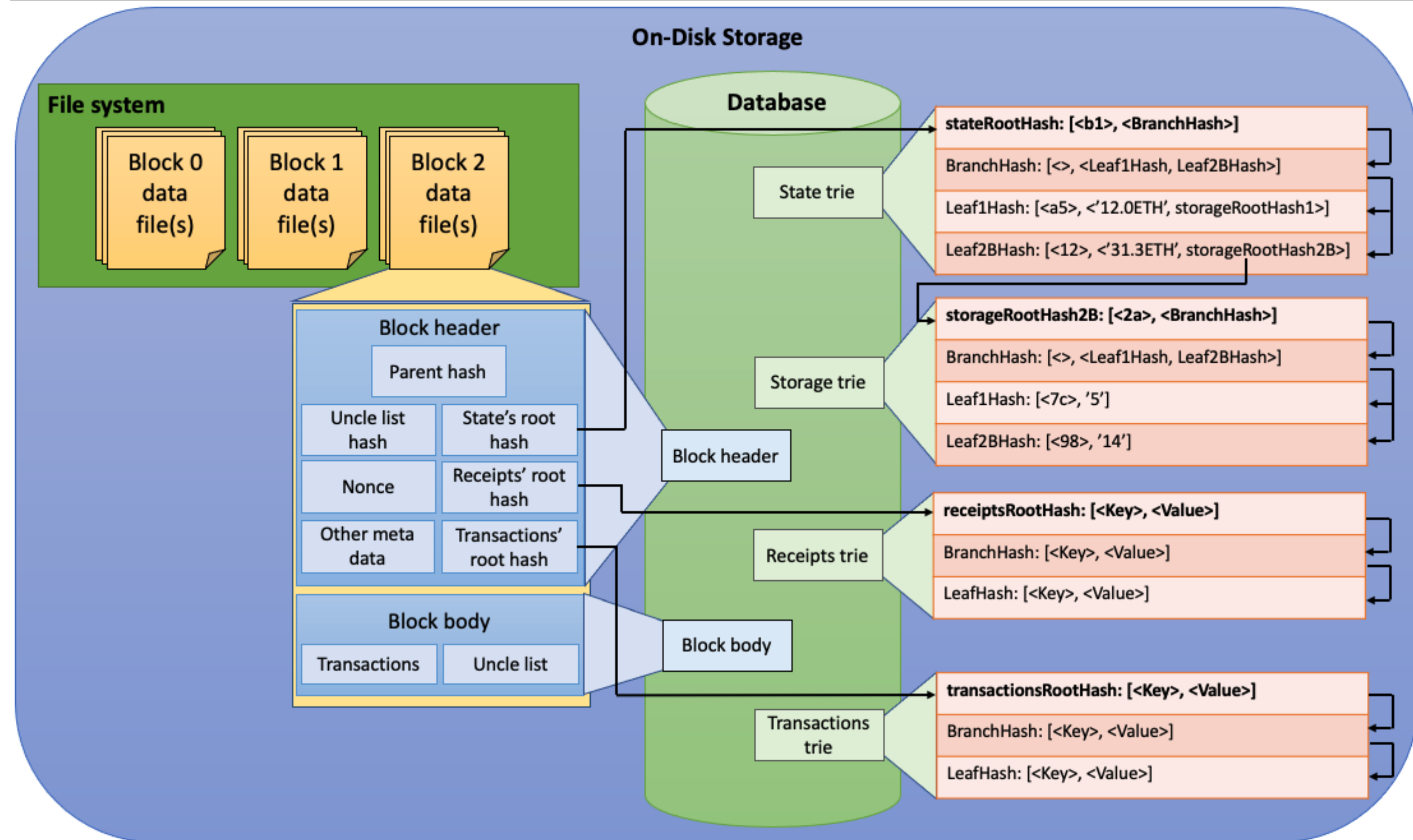
State trie

Stores accounts:

Address:
[Value,
Nonce,
StorageRoot,
CodeHash]

Trie:
Merkle tree that supports

update
lookup
proof



StorageRoot is the root of a different trie.

Ethereum

State trie

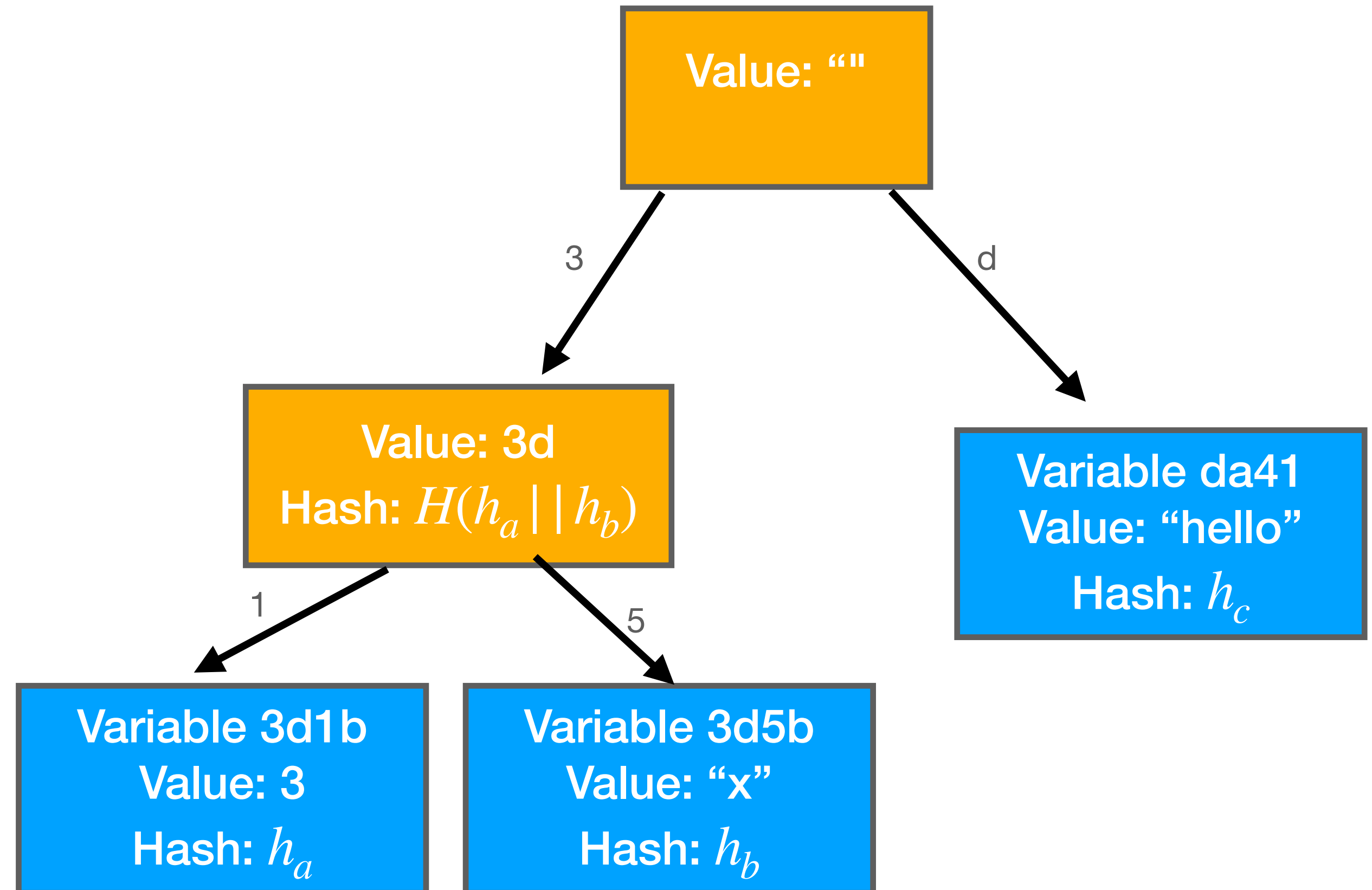
Stores accounts:

Address:
[Value,
Nonce,
StorageRoot,
CodeHash]

Trie:
Merkle tree that supports

update
lookup
proof

StorageRoot is the root of a different trie.



Ethereum

State trie

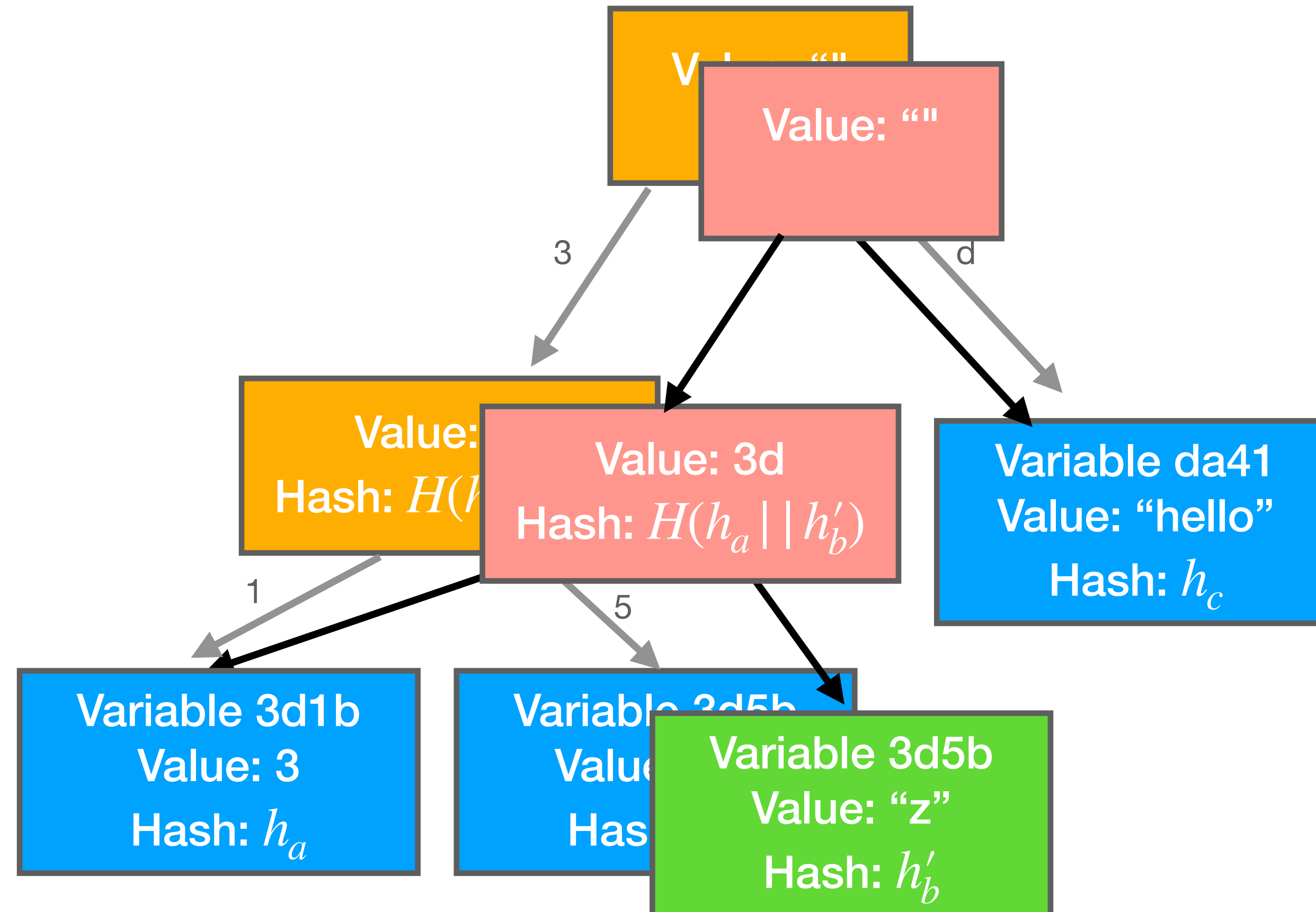
Stores accounts:

Address:
[Value,
Nonce,
StorageRoot,
CodeHash]

Trie:
Merkle tree that supports

update
lookup
proof

StorageRoot is the root of a different trie.



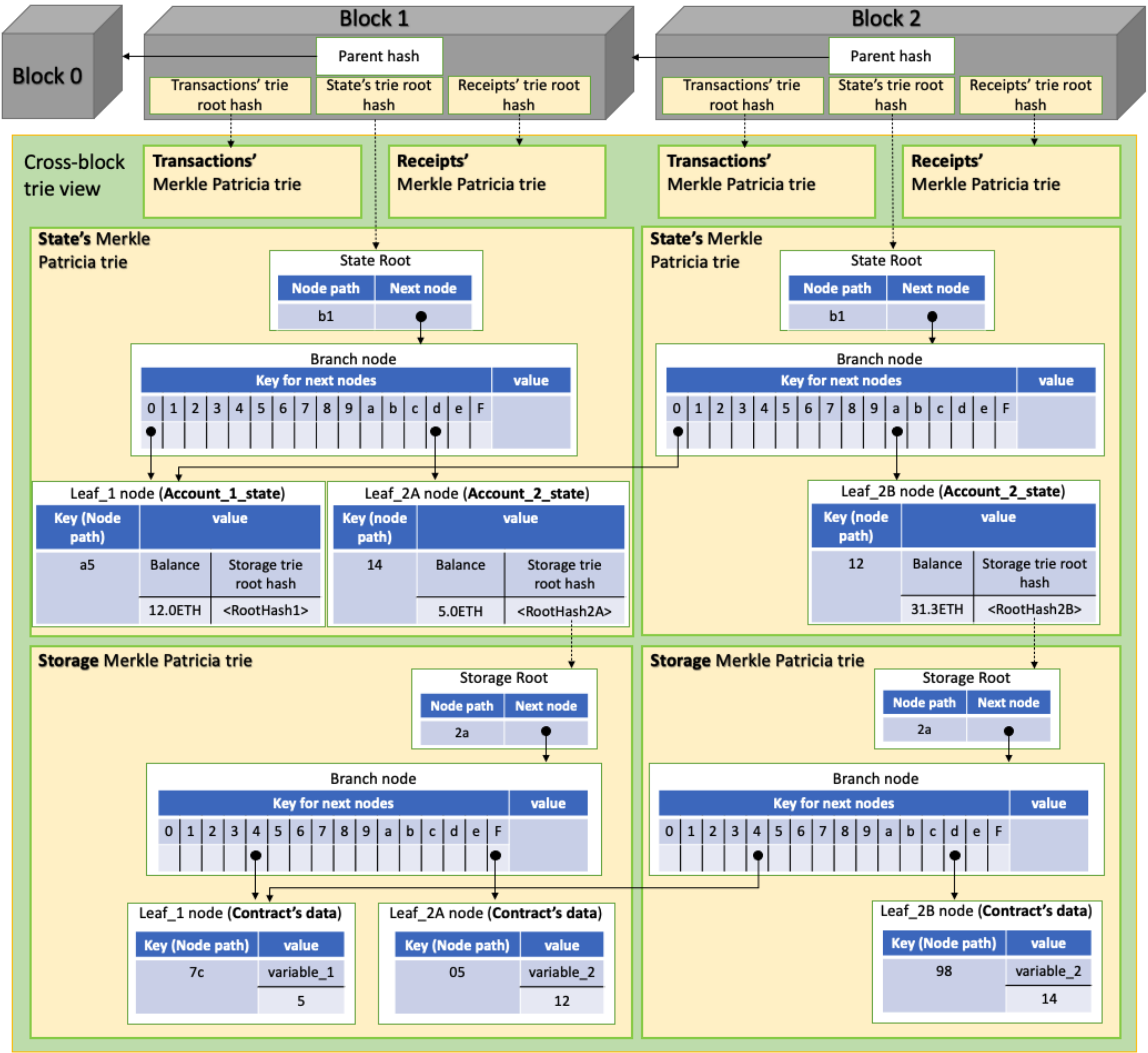
If a variable gets updated, new nodes get created on the path to root.

Ethereum

State trie

Reuses storage

Allows roleback
and forks



Ethereum

Read contract state

1. ask trusted node
2. receive inclusion proof for

stateRoot: storageTrie
account state: stateTrie

and block header

Fees and Gas

Ethereum

Gas

How to pay transaction fees in Ethereum?

- all bytecode instructions have a cost specified in Gas
- transaction has fixed cost in Gas
- especially: storing values is expensive

Why specific gas per instruction:

- An infinite loop will cost infinitely much gas -> avoid denial of service

Ethereum

Gas

Each transaction can have a different exchange rate *Gas to Ether*

- *Old way*: specify *gas price* and *gas limit*
- *New way*: specify *tip (priority fee)*, *max fee per gas* and *gas limit*

Old way specify *Gas price* and *Gas limit*

- *Gas price* is ether given per gas
- *Gas limit* is how much the transaction may spend at most

Ethereum

Gas

Each transaction can have a different exchange rate *Gas to Ether*

- *Old way*: specify *gas price* and *gas limit*
- *New way*: specify *tip (priority fee)*, *max fee per gas* and *gas limit*

New way specify *tip* and *max fee* (EIP-1559)

- *Gas price* is calculated $tip + base_fee$
- *Gas limit* is how much the transaction may spend at most

Ethereum

Gas

Each transaction can have a different exchange rate *Gas to Ether*

- *New way*: specify *tip* (priority fee), *max fee per gas* and *gas limit*

New way specify *tip* and *max fee* (EIP-1559)

- *gas price* is calculated $tip + base_fee$
- *base_fee* is given by protocol, the same for all transactions in a block
- *max_fee*: do not execute if $base_fee > max_fee - tip$
- *gas limit* is how much the transaction may spend at most

Ethereum

Gas

Gas Used	Gas Limit	Base Fee
28,593,288 (47.66%)	60,000,000	0.064 Gwei
24,485,633 (40.81%)	60,000,000	0.065 Gwei
46,074,987 (76.79%)	59,999,943	0.061 Gwei
24,909,707 (41.56%)	59,941,408	0.062 Gwei
27,580,444 (45.97%)	60,000,000	0.063 Gwei
54,381,329 (90.64%)	60,000,000	0.057 Gwei
27,364,250 (45.61%)	60,000,000	0.058 Gwei
23,418,256 (39.03%)	60,000,000	0.059 Gwei

Gas Used	Gas Limit	Base Fee
23,063,456 (38.48%)	59,941,408	0.163 Gwei
54,068,920 (90.11%)	60,000,000	0.148 Gwei
10,700,502 (17.83%)	60,000,000	0.161 Gwei
27,552,861 (45.92%)	59,999,943	0.163 Gwei
34,826,585 (58.10%)	59,941,408	0.159 Gwei
39,282,593 (65.47%)	60,000,000	0.153 Gwei
27,063,519 (45.11%)	60,000,000	0.155 Gwei
37,904,066 (63.17%)	60,000,000	0.15 Gwei

Gas price = tip💵👷 + base fee🔥

Ethereum

Gas

What happens if you hit the *Gas limit*?

- Exception is thrown and transaction reverted.
- Gas is still payed!

Which transactions are included?

- Miners will include transactions offering the highest gas price.
- Blocks have maximum amount of gas.

Smart Contract Security

Smart Contract Security

- Prevent exploits and attacks
- Attacks can:
 - Steal tokens
 - Leave contract disfunctional

Problem:

- Code and smart contract state are public

If it is standing around in public, and it can be easily broken, someone will eventually break it.

Smart Contract Security

Problem:

- Code and smart contract state are public
- Can try exploit before deploying it (in development environment)
- Can automatically scan for possible exploits

Smart Contract Security

Problem:

- Smart contract cannot be upgraded
 - TDD, Formal verification, code audits, bug bounties
 - Language and platform support is constantly improving

Known vulnerabilities

Smart Contract Security

Known vulnerabilities

- Integer overflow
- Force money to contract
- Re-entrancy

Smart Contract Security

Integer overflow

Increment an integer above its max number

- Solidity has 256 bit uint
- **Overflow:** Incrementing to 2^{256} gives 0



Smart Contract Security

Integer overflow

TimeLock example

- Funds can be taken out only after 1 week
- How to exploit?

```
contract TimeLock {  
  
    mapping(address => uint) public balances;  
    mapping(address => uint) public lockTime;  
  
    function deposit() public payable {  
        balances[msg.sender] += msg.value;  
        lockTime[msg.sender] = now + 1 weeks;  
    }  
  
    function increaseLockTime(uint _secondsToIncrease)  
        public {  
        lockTime[msg.sender] += _secondsToIncrease;  
    }  
  
    function withdraw() public {  
        require(balances[msg.sender] > 0);  
        require(now > lockTime[msg.sender]);  
        uint balance = balances[msg.sender];  
        balances[msg.sender] = 0;  
        msg.sender.transfer(balance);  
    }  
}
```

Smart Contract Security

Integer overflow

TimeLock example

- Funds can be taken out only after 1 week
- Use `increaseLockTime` to increase lock time to 0.

Fixed when using new Solidity version!

```
contract TimeLock {  
  
    mapping(address => uint) public balances;  
    mapping(address => uint) public lockTime;  
  
    function deposit() public payable {  
        balances[msg.sender] += msg.value;  
        lockTime[msg.sender] = now + 1 weeks;  
    }  
  
    function increaseLockTime(uint _secondsToIncrease)  
        public {  
        lockTime[msg.sender] += _secondsToIncrease;  
    }  
  
    function withdraw() public {  
        require(balances[msg.sender] > 0);  
        require(now > lockTime[msg.sender]);  
        uint balance = balances[msg.sender];  
        balances[msg.sender] = 0;  
        msg.sender.transfer(balance);  
    }  
}
```

Smart Contract Security

Integer overflow

Token example

- How to exploit this?

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity 0.5.11;
3
4  contract Token {
5      mapping(address => uint256) balances;
6      uint256 public totalSupply;
7
8      constructor(uint256 _initialSupply) public {
9          balances[msg.sender] = totalSupply = _initialSupply;
10     }
11
12     function transfer(address _to, uint256 _value) public returns (bool) {
13         require(balances[msg.sender] - _value >= 0);
14         balances[msg.sender] -= _value;
15         balances[_to] += _value;
16         return true;
17     }
18
19     function balanceOf(address _owner) public view returns (uint256 balance) {
20         return balances[_owner];
21     }
22 }
```

Smart Contract Security

Re-entrancy

Vulnerability when sending money to a different contract

- Sending money can trigger a function

```
msg.sender.call.value(_weiToWithdraw)("");  
if (success){  
    balances[msg.sender] -= _weiToWithdraw;  
}
```

- This function can re-invoke the current function

```
// fallback function - where the magic happens  
function () external payable {  
    if (address(etherStore).balance >= 1 ether) {  
        etherStore.withdrawFunds(1 ether);  
    }  
}
```

Owner with balance 1eth can withdraw all contract balance.

Smart Contract Security

Re-entrancy

Vulnerability when sending money to a different contract

```
// vulnerable function
function withdrawFunds(uint256 _weiToWithdraw) public {
    require(balances[msg.sender] >= _weiToWithdraw);
    (bool success, ) = msg.sender.call.value(_weiToWithdraw)("");
    if (success) {
        balances[msg.sender] -= _weiToWithdraw;
    }
}
```

```
// vulnerable
withdraw()
- check
- send money using call
- if success, update
```

Smart Contract Security

Re-entrancy

Vulnerability when sending money to a different contract

```
// fallback function – where the magic happens
function() external payable {
    if (address(etherStore).balance >= 1 ether) {
        etherStore.withdrawFunds(wtype, 1 ether);
    }
}
```

```
// attack
default()
– check victim has more money
– call withdraw
```

Smart Contract Security

Re-entrancy

```
// vulnerable  
withdraw()  
- check  
- send money using call
```

```
// attack  
default()  
- check victim has more money  
- call withdraw
```

```
// vulnerable  
withdraw()  
- check  
- send money using call
```

```
// attack  
default()  
- check victim has more money  
- call withdraw
```

```
- if success, update  
- if success, update
```

Update happens after recursive call.

Smart Contract Security

Re-entrancy

Vulnerability when sending money to a different contract

Mitigation

- Pattern: Update, then send money

```
// vulnerable
withdraw()
- check
- update
- send money using call
- if not success, revert
```

Smart Contract Security

Re-entrancy

```
// vulnerable
withdraw()
- check
- Update
- send money using call
```

```
// attack
default()
- check victim has more money
- call withdraw
```

```
// vulnerable
withdraw()
- check
- Update
- send money using call
```

check fails here

```
- if success, update
```

```
- if success, update
```

Smart Contract Security

Re-entrancy

Vulnerability when sending money to a different contract

Mitigation

- Pattern: Update, then send money
- Outdated:
Use **send** or **transfer** not call

Method	address.send()	address.transfer()	address.call.value()()
Possibility to set gas limit	No	No	Yes
limited gas	2300	2300	Settable
Return value when error	FALSE	Throws exception	FALSE

Smart Contract Security

Re-entrancy

Vulnerability when sending money to a different contract

limited gas

Mitigation

- Pattern: Update, then send money
- Outdated:
Use **send** or **transfer** not call
- can only be used to send to EOA
- not future proof
(gas cost may change)

Method	address.send()	address.transfer()	address.call.value()()
Possibility to set gas limit	No	No	Yes
Gas limit	2300	2300	Settable
Return value when error	FALSE	Throws exception	FALSE

Smart Contract Security

Forcing Ether

- Ether can be send to an address without you wanting it.
- Using *selfdestruct*, or sending before contract creation
- Sending money to address before contract is created

Mitigation

- Use a balance variable

Example

- EtherGame

Smart Contract Security

Forcing Ether

- Ether can be send to an address without you wanting it.
- Using *selfdestruct*
- Sending money to address before contract is created

Mitigation

- Use a balance variable

Example

- EtherGame

Smart Contract Security

Other considerations

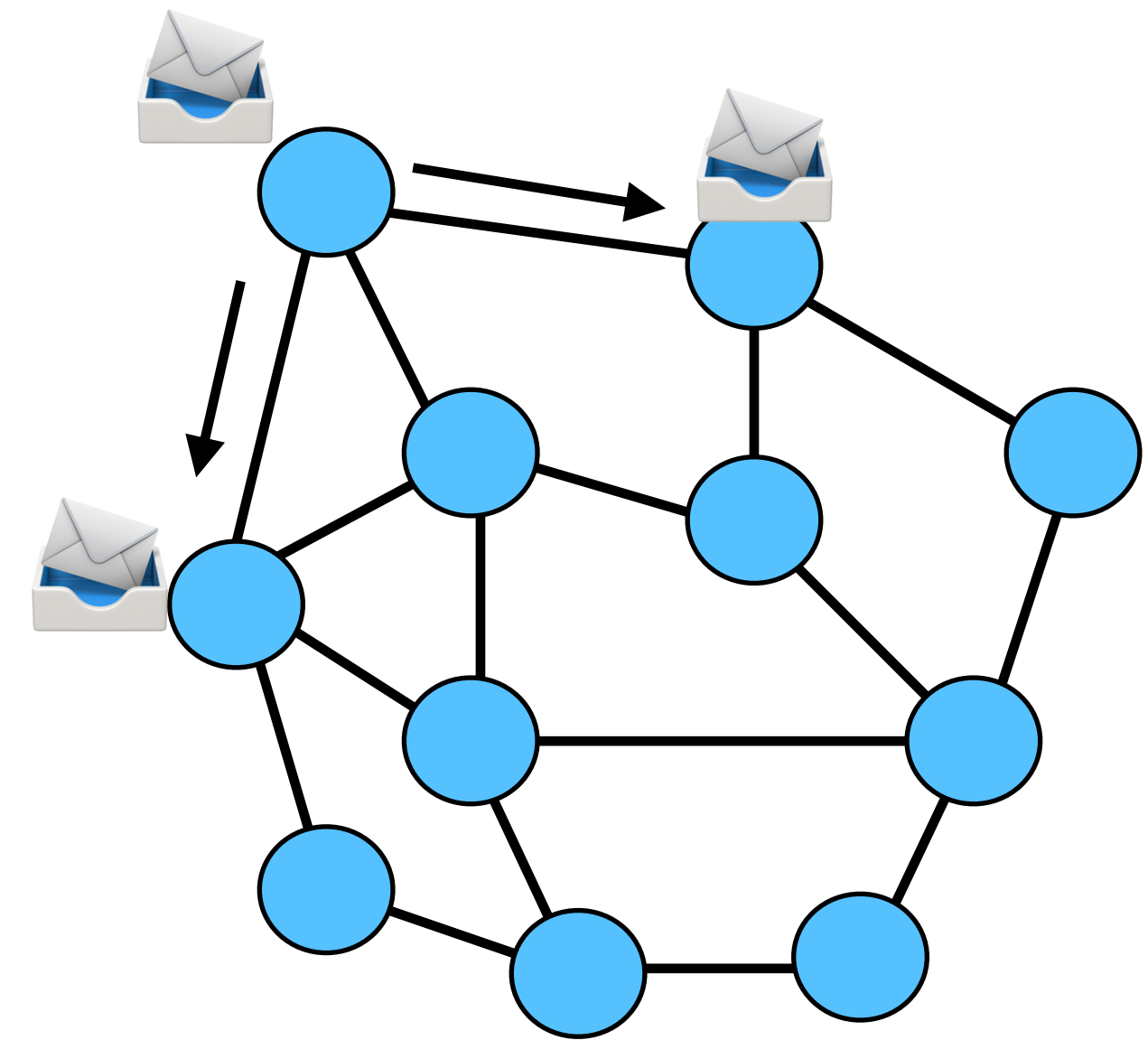
- **Randomness** is difficult to get
Can be influenced by miners
- **Timestamp** can be influenced
- All values are **public**
- Set helper function private
avoid being called directly
- Check library addresses

Frontrunning

Smart Contract Security

Frontrunning

- Transaction submission:
 - Submitted to one node
 - Broadcast through the network

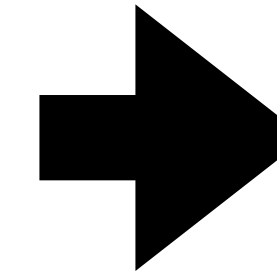
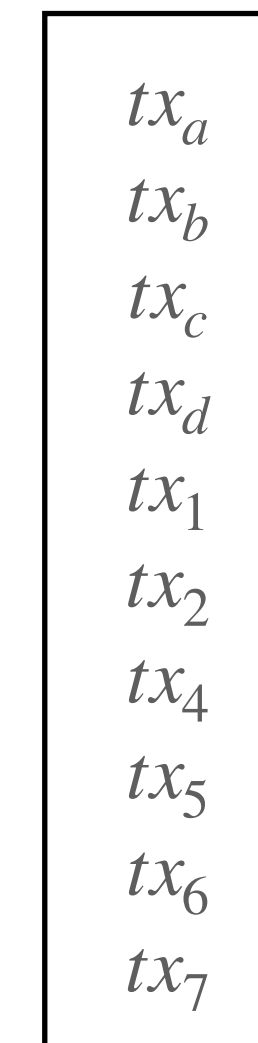


Smart Contract Security

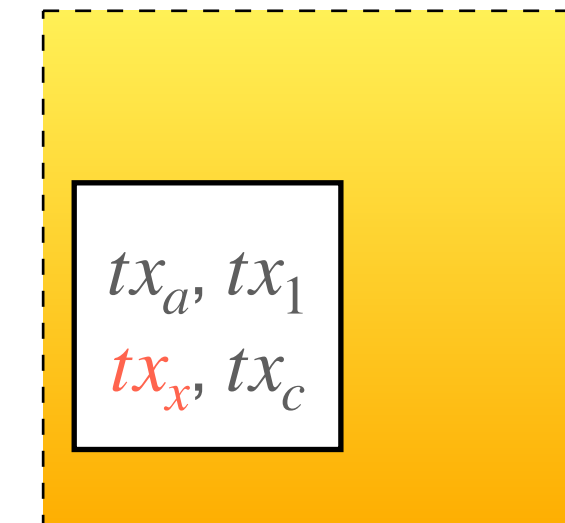
Frontrunning

- Transaction submission:
 - Submitted to one node
 - Broadcast through the network
 - Proposer decides which transaction to include in a block
 - can decide order
 - can add more transactions

Mempool



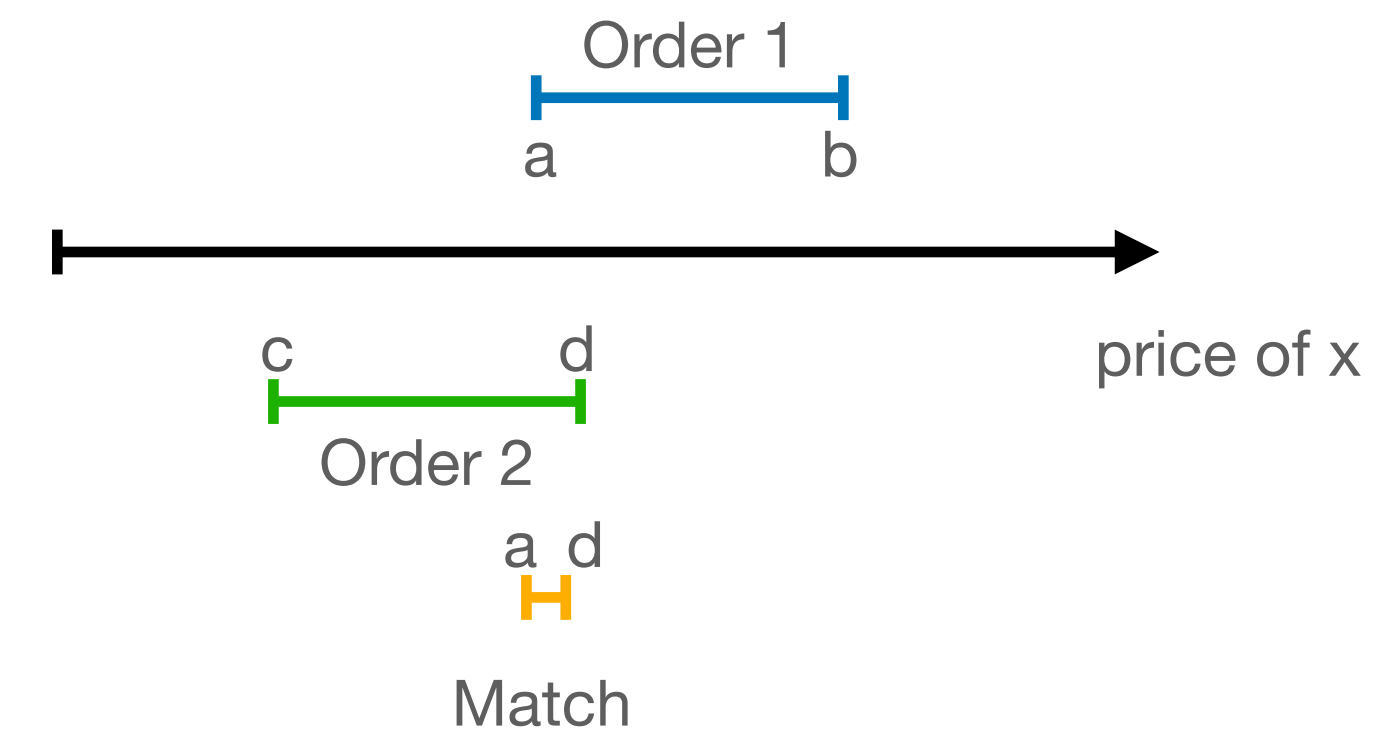
Block



Smart Contract Security

Frontrunning

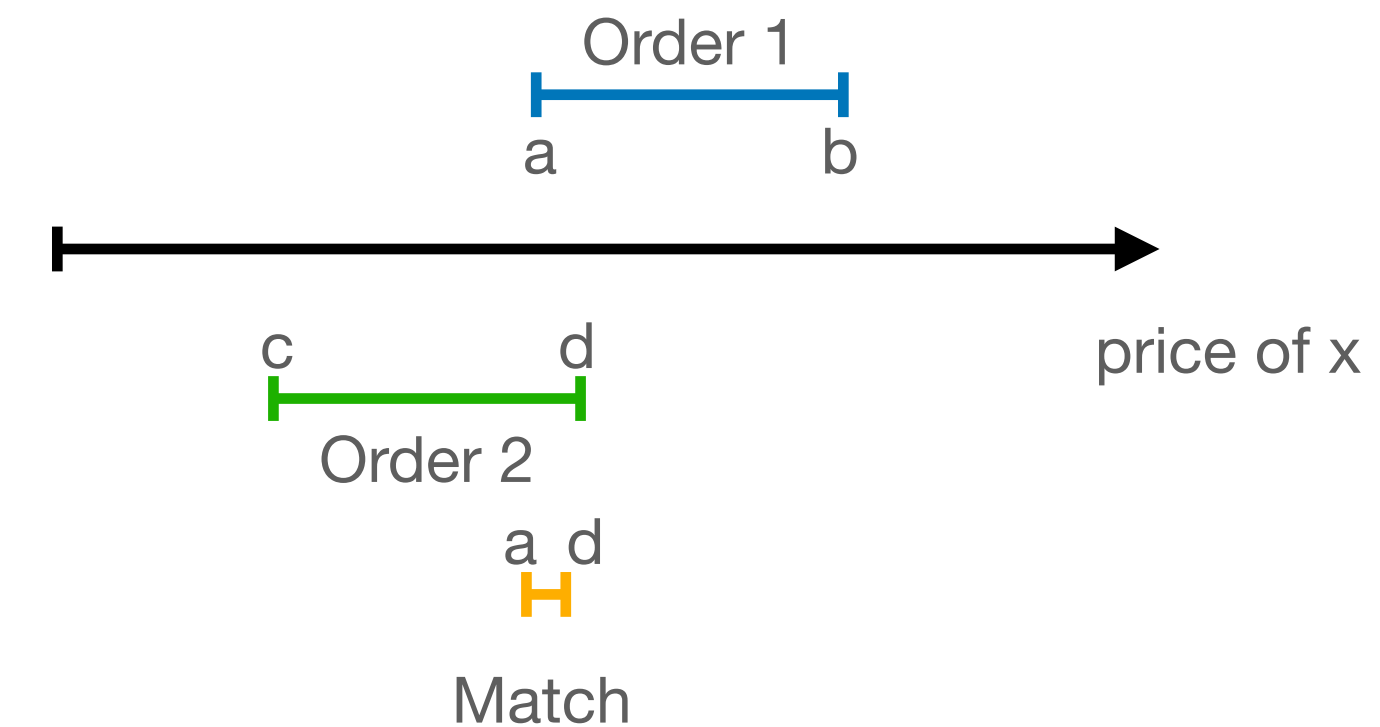
- Example:
 - Order1:
Sell x at price [a,b]
 - Order 2:
Buy x at price [c,d]



Smart Contract Security

Frontrunning

- Example:
 - Order1:
Sell x at price [a,b]
 - Order x:
Buy x at price [a]
 - Order y:
Sell x at price [d]
 - Order 2:
Buy x at price [c,d]



Smart Contract Security

Frontrunning

- Example2:
 - Order:
Buy 100.000 tokens
 - This order increases the price.
- Attack:
 - Order x: Buy 100 tokens
 - Order:
Buy 100.000 tokens
 - This order increases the price.
 - Order y: Sell 100 tokens

Smart Contract Security

Frontrunning

- Defense:
 - Sometimes: commit-reveal
 - Protection services:
Send transaction privately
to nodes, that promise not
to frontrun you.

Commit Reveal

Commit-Reveal

A pattern to avoid revealing information early

Commit Reveal

- First transaction:
Store a hash of your value in a smart contract.
- Second transaction:
Reveal value

Concerns:

1. Include random salt in hash to avoid guessing
2. Check that hash is correct on reveal
3. Handle missing reveal

Commit-Reveal

A pattern to avoid revealing information early

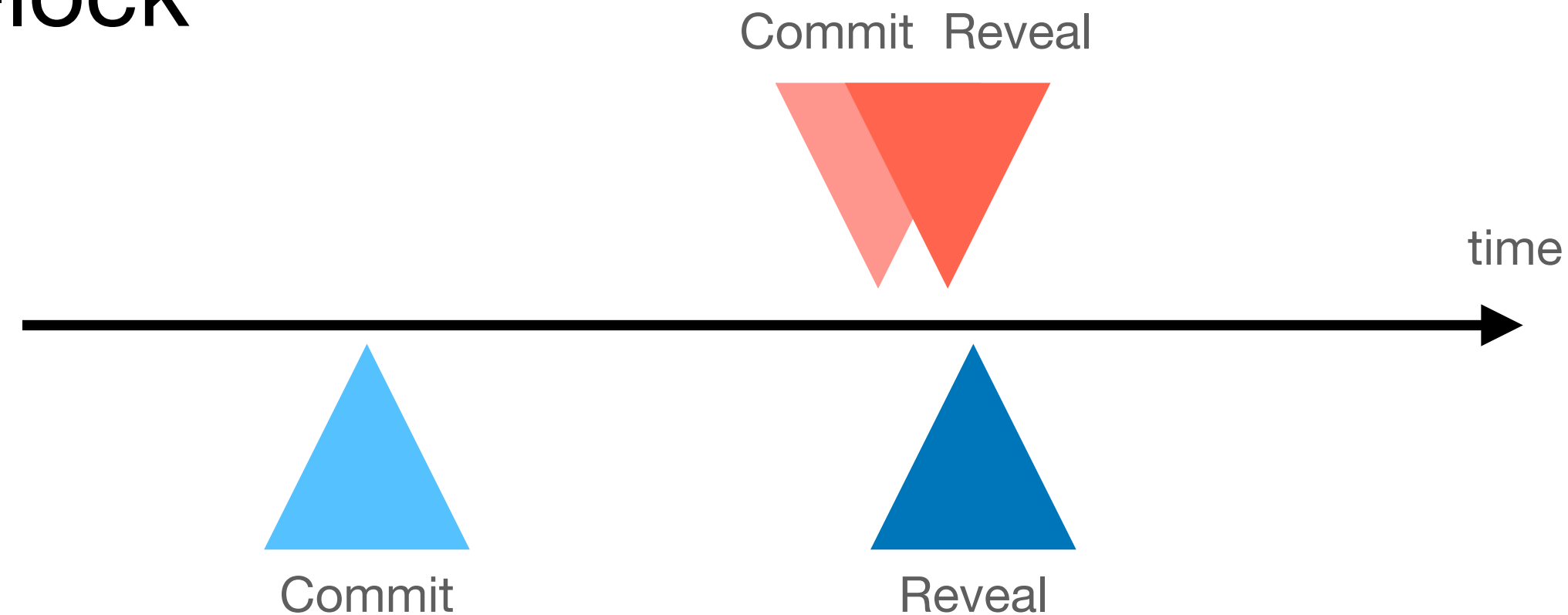
Can commit-reveal help against front-running?

Commit-Reveal

A pattern to avoid revealing information early

Can commit-reveal help against front-running?

- Needs to be supported by Smart-Contract
- Need time-lock

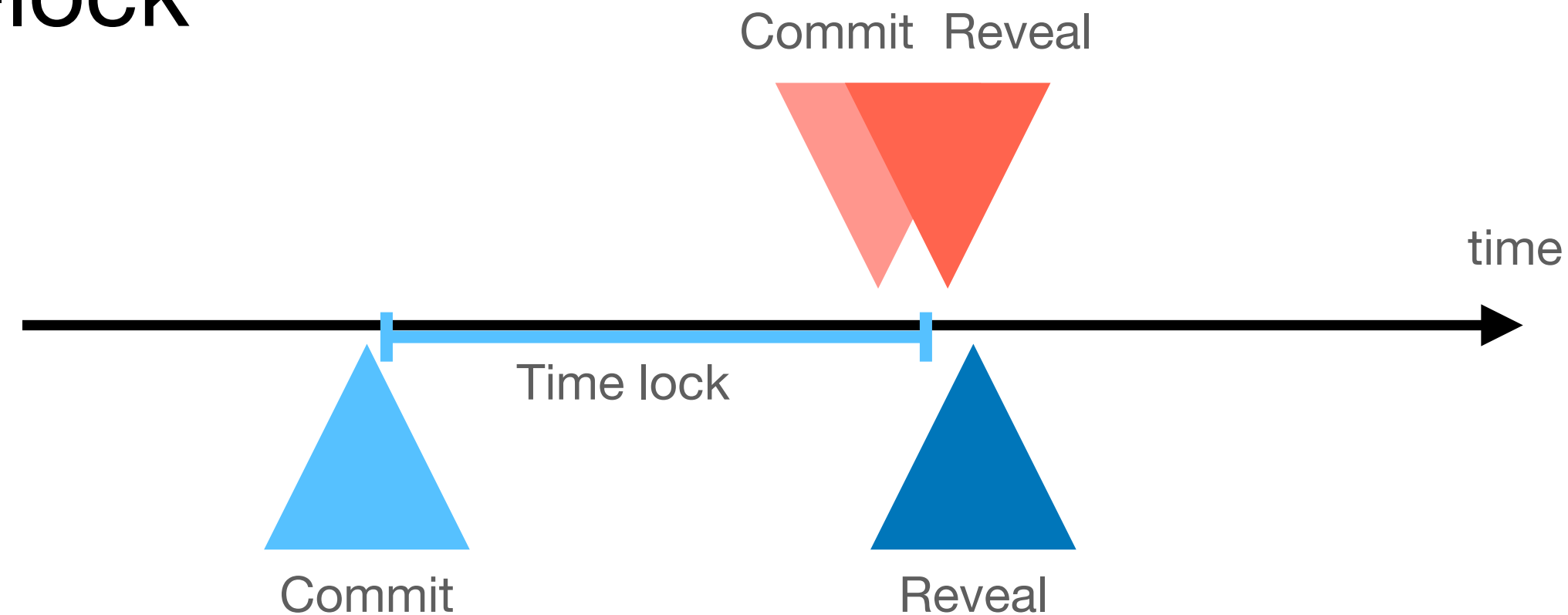


Commit-Reveal

A pattern to avoid revealing information early

Can commit-reveal help against front-running?

- Needs to be supported by Smart-Contract
- Need time-lock



Commit-Reveal

A pattern to avoid revealing information early

Can commit-reveal help against front-running?

- Needs to be supported by Smart-Contract
- Need time-lock

